Lab 3.1 — Function Definition & Calling in C/C++

Deney 3.1 — C/C++'ta Fonksiyon Tanımlama ve Çağırma

实验 3.1 — 函数的定义与调用

Shíyàn 3.1 — Hánshù de dìngyì yǔ diàoyòng

Trilingual study guide: English · Türkçe · 中文 (with Pinyin)

What was in the archive / Arşivde ne vardı / 压缩包内容	**Notes**
test/main.cpp	**The only file that matters** — the lab source code + instructions (in Chinese).
test.sln, test.vcxproj, *.filters, *.user	Visual Studio 2010 solution/project files.
Debug/, ipch/, *.sdf, *.suo, *.pdb, *.ilk, *.tlog	Build output & IDE cache — auto-generated, can be ignored/deleted.

1. The lab instructions (original) / Deney talimatları / 实验要求

Task 1 / Görev 1 / 要求 1

EN Because of the order of function definition and call, the code below does **not compile**. Read the error message and fix it in **two ways**.

TR Fonksiyon tanımı ve çağrı sırası yüzünden aşağıdaki kod **derlenmez**. Hata mesajını inceleyin ve **iki yöntemle** çözün.

中文 因为函数定义与调用顺序问题，以下代码无法编译通过，观察出错信息，使用两种方法解决

Yīnwèi hánshù dìngyì yǔ diàoyòng shùnxù wèntí, yǐxià dàimǎ wúfǎ biānyì tōngguò, guānchá chūcuò xìnxī, shǐyòng liǎng zhǒng fāngfǎ jiějué.

Task 2 / Görev 2 / 要求 2

EN Notice the **warnings** produced while compiling; remove them, e.g. by changing data types.

TR Derleme sırasında oluşan **uyarılara** dikkat edin; veri tiplerini değiştirerek uyarıları giderin.

中文 注意编译过程中产生的警告，通过改变数据类型定义等方法消除警告

Zhùyì biānyì guòchéng zhōng chǎnshēng de jǐnggào, tōngguò gǎibiàn shùjù lèixíng dìngyì děng fāngfǎ xiāochú jǐnggào.

Task 3 / Görev 3 / 要求 3

EN Debug using **F10 (Step Over)** and **F11 (Step Into)**, and compare the difference.

TR **F10 (Step Over / üzerinden geç)** ve **F11 (Step Into / içine gir)** ile hata ayıklayın, farkı karşılaştırın.

中文 使用F10逐过程和F11逐语句两种方法调试，比较其区别

Shǐyòng F10 zhú guòchéng hé F11 zhú yǔjù liǎng zhǒng fāngfǎ tiáoshì, bǐjiào qí qūbié.

Task 4 / Görev 4 / 要求 4

EN Understand **variable scope**; rewrite the program so that a sub-function (func2) can **change the variable c in main**.

TR **Değişken kapsamını (scope)** anlayın; programı, bir alt fonksiyonun (func2) **main içindeki c değişkenini değiştirebileceği** şekilde yeniden yazın.

中文 初步理解变量的作用域问题，改写程序，使子程序func2能够更改主程序中的变量c

Chūbù lǐjiě biànliàng de zuòyòngyù wèntí, gǎixiě chéngxù, shǐ zǐ chéngxù func2 nénggòu gēnggǎi zhǔ chéngxù zhōng de biànliàng c.

2. Original code (as given) / Orijinal kod / 原始代码

```
//double foo1();    //函数的声明，返回值为浮点型
//double foo2(double a, double b);  //函数的声明，含有两个参数（形式参数，不产生实际内存分配），返回值为浮点型
//函数定义语句如果在调用语句之前，可以省略声明语句
//函数定义语句如果在调用语句之后，必须提前用声明语句提示编译器存在该函数，以及该函数的类型、参数等
//函数声明语句与函数定义语句语法一致，但没有 {} 部分和其内部的具体代码实现

//int c; //在所有函数外声明的变量，为全局变量，可被本文件中所有函数访问
int main()
{
    int c;//函数内声明的变量，为局部变量，只在函数内部访问
    c = 1;
    double  d = 1.4;
    foo1();
    //函数的调用,忽略返回值。如果函数没有返回值，最好定义为void类型
    c = foo2(10,20);
    //函数的调用,将实际参数填写到形式参数对应的位置,接收返回值的变量类型要与返回值类型一致，类型不一致会进行转换并会产生警告
    d = foo2(1,2);
    d = c;//为了调试中方便观察c的值
    return 0;
}
double foo1()       //函数的定义,，如果函数没有返回值，最好定义为void类型
{
    return 1.0;
}
double foo2(double a, double b)     //函数的定义，与函数声明一致
{
    int c;
    c=100;  //变量作用域问题：局部变量是否影响主函数中的同名变量？
    return (a+b);   //可利用return返回计算结果，结果类型应与定义类型一致，否则会进行
}
```

EN Compiling gives: `error: 'foo1' was not declared in this scope` (MSVC: `C3861: identifier not found`). The compiler reads top-to-bottom, and main() uses foo1/foo2 before it has seen them.

TR Derleyici şu hatayı verir: `'foo1' bu kapsamda bildirilmedi` (MSVC: C3861). Derleyici kodu yukarıdan aşağıya okur; main() foo1/foo2'yi henüz görmeden kullanıyor.

中文 编译报错：找不到标识符 foo1/foo2。编译器从上往下读代码，main() 在看到函数之前就调用了它们。

Biānyì bàocuò: zhǎo bú dào biāozhìfú foo1/foo2. Biānyìqì cóng shàng wǎng xià dú dàimǎ, main() zài kàndào hánshù zhīqián jiù diàoyòng le tāmen.

3. What you MUST know / Bilmeniz gerekenler / 必须掌握的知识点

3.1 Declaration vs. Definition / Bildirim ve Tanım / 声明与定义

EN A **declaration** (prototype) tells the compiler the name, return type and parameter types: `double foo2(double a, double b);` — it ends with **;** and has **no { } body**. A **definition** contains the body with the real code.

TR **Bildirim** (prototip) derleyiciye adı, dönüş tipini ve parametre tiplerini söyler; **;** ile biter ve **{ } gövdesi yoktur**. **Tanım** ise asıl kodu içeren gövdedir.

中文

函数声明告诉编译器函数名、返回类型和参数类型，以分号结尾，没有 {} 部分；函数定义包含具体的代码实现。

> *Hánshù shēngmíng gàosù biānyìqì hánshù míng, fǎnhuí lèixíng hé cānshù lèixíng, yǐ fēnhào jiéwěi, méiyǒu {} bùfen; hánshù dìngyì bāohán jùtǐ de dàimǎ shíxiàn.*

3.2 The ordering rule / Sıralama kuralı / 顺序规则

EN If the definition comes **before** the call, the declaration can be omitted. If it comes **after** the call, you **must** declare it first.

TR Tanım çağrıdan **önce** ise bildirim gerekmez. Tanım çağrıdan **sonra** ise önce **mutlaka** bildirim yapılmalıdır.

中文

函数定义语句如果在调用语句之前，可以省略声明语句；如果在调用语句之后，必须提前用声明语句提示编译器。

> *Hánshù dìngyì yǔjù rúguǒ zài diàoyòng yǔjù zhīqián, kěyǐ shěnglüè shēngmíng yǔjù; rúguǒ zài diàoyòng yǔjù zhīhòu, bìxū tíqián yòng shēngmíng yǔjù tíshì biānyìqì.*

3.3 Formal vs. actual parameters / Biçimsel ve gerçek parametreler / 形式参数与实际参数

EN **Formal parameters** are the names in the function header (`a, b`) — in a declaration they allocate no memory. **Actual parameters (arguments)** are the values you pass at the call (`10, 20`). Values are **copied** (pass-by-value).

TR **Biçimsel parametreler** fonksiyon başlığındaki adlardır (`a, b`). **Gerçek parametreler (argümanlar)** çağrıda verilen değerlerdir (`10, 20`). Değerler **kopyalanır** (değerle aktarım).

中文 形式参数是函数头中的参数名（a, b），声明中不产生实际内存分配；实际参数是调用时填入的值（10, 20），按值传递，会被复制。

> *Xíngshì cānshù shì hánshù tóu zhōng de cānshù míng (a, b), shēngmíng zhōng bù chǎnshēng shíjì nèicún fēnpèi; shíjì cānshù shì diàoyòng shí tiánrù de zhí (10, 20), àn zhí chuándì, huì bèi fùzhì.*

3.4 Return value & type conversion / Dönüş değeri ve tip dönüşümü / 返回值与类型转换

EN `return` sends a result back. The receiving variable should have the **same type**. `int c = foo2(10,20);` converts double→int and gives warning **C4244: possible loss of data**. A function that returns nothing should be `void`.

TR `return` sonucu geri gönderir. Alan değişken **aynı tipte** olmalıdır. `int c = foo2(10,20);` double→int dönüşümü yapar ve **C4244: veri kaybı olabilir** uyarısı verir. Değer döndürmeyen fonksiyon `void` olmalıdır.

中文 可利用return返回计算结果，接收返回值的变量类型要与返回值类型一致，类型不一致会进行转换并产生警告。没有返回值的函数最好定义为void类型。

> *Kě lìyòng return fǎnhuí jìsuàn jiéguǒ, jiēshōu fǎnhuízhí de biànliàng lèixíng yào yǔ fǎnhuízhí lèixíng yízhì, lèixíng bù yízhì huì jìnxíng zhuǎnhuàn bìng chǎnshēng jǐnggào. Méiyǒu fǎnhuízhí de hánshù zuìhǎo dìngyì wéi void lèixíng.*

3.5 Local vs. global variables (scope) / Yerel ve global değişkenler (kapsam) / 局部变量与全局变量（作用域）

EN A variable declared **outside all functions** is **global** — every function in the file can use it. A variable declared **inside a function** is **local** — only that function sees it. The `int c` inside foo2 is a **different variable** from c in main: setting it to 100 does **not** change main's c.

TR **Tüm fonksiyonların dışında** bildirilen değişken **global**dir; dosyadaki her fonksiyon erişebilir. **Fonksiyon içinde** bildirilen **yerel**dir; sadece o fonksiyon görür. foo2 içindeki c, main'deki c'den **farklı bir değişkendir**; 100 yapmak main'dekini **değiştirmez**.

中文 在所有函数外声明的变量为全局变量，可被本文件中所有函数访问；函数内声明的变量为局部变量，只在函数内部访问。foo2 中的 c 与主函数中的同名变量 c 互不影响。

> *Zài suǒyǒu hánshù wài shēngmíng de biànliàng wéi quánjú biànliàng, kě bèi běn wénjiàn zhōng suǒyǒu hánshù fǎngwèn; hánshù nèi shēngmíng de biànliàng wéi júbù biànliàng, zhǐ zài hánshù nèibù fǎngwèn. foo2 zhōng de c yǔ zhǔ hánshù zhōng de tóngmíng biànliàng c hù bù yǐngxiǎng.*

3.6 Debugging: F10 vs F11 / Hata ayıklama / 调试

EN **F10 = Step Over**: runs a whole function call as one step (does not enter it). **F11 = Step Into**: enters the called function and goes line by line inside it. Use F9 to set a breakpoint; watch variables in the Locals/Watch window.

TR **F10 = Step Over**: fonksiyon çağrısını tek adımda çalıştırır (içine girmez). **F11 = Step Into**: çağrılan fonksiyonun içine girer, satır satır ilerler. F9 ile kesme noktası (breakpoint) koyun; değişkenleri Locals/Watch penceresinde izleyin.

中文 F10 逐过程：把整个函数调用当作一步执行，不进入函数内部；F11 逐语句：进入被调用的函数内部，一条一条语句执行。F9 设置断点。

> *F10 zhú guòchéng: bǎ zhěnggè hánshù diàoyòng dàngzuò yí bù zhíxíng, bú jìnrù hánshù nèibù; F11 zhú yǔjù: jìnrù bèi diàoyòng de hánshù nèibù, yì tiáo yì tiáo yǔjù zhíxíng. F9 shèzhì duàndiǎn.*

4. Solutions / Çözümler / 解答

Task 1 — Method A: add declarations (prototypes) before main

EN Un-comment the two prototype lines at the top.

TR En üstteki iki prototip satırının yorumunu kaldırın.

中文 方法一：在 main 之前加上函数声明。

Fāngfǎ yī: zài main zhīqián jiā shàng hánshù shēngmíng.

```
double foo1();                    // declaration / bildirim / 声明
double foo2(double a, double b);  // declaration

int main() { ... foo1(); c = foo2(10,20); ... }

double foo1() { return 1.0; }                    // definition / tanım / 定义
double foo2(double a, double b) { return a + b; }
```

Task 1 — Method B: move the definitions above main

EN Put foo1 and foo2 before main(); then no declaration is needed.

TR foo1 ve foo2'yi main()'in üstüne taşıyın; bildirim gerekmez.

中文 方法二：把函数定义移到 main 之前，就可以省略声明。

Fāngfǎ èr: bǎ hánshù dìngyì yí dào main zhīqián, jiù kěyǐ shěnglüè shēngmíng.

```
double foo1() { return 1.0; }
double foo2(double a, double b) { return a + b; }

int main() { ... }   // now foo1/foo2 are already known
```

Task 2 — Remove the warnings

EN Warning comes from `c = foo2(10,20);` (double→int). Fixes: make c a `double`, or cast explicitly `c = (int)foo2(10,20);`, or make foo2 return `int`. Also make foo1 `void` because its return value is ignored.

TR Uyarı `c = foo2(10,20);` satırından gelir (double→int). Çözüm: c'yi `double` yapın, ya da açık dönüşüm `(int)` kullanın, ya da foo2'nin dönüş tipini `int` yapın. foo1'in değeri kullanılmadığı için `void` yapın.

中文 警告来自 c = foo2(10,20)（double 转 int）。消除方法：把 c 定义为 double，或者使用强制类型转换 (int)，或者把 foo2 的返回类型改为 int。foo1 的返回值没有被使用，可以改成 void。

Jǐnggào láizì c = foo2(10,20) (double zhuǎn int). Xiāochú fāngfǎ: bǎ c dìngyì wéi double, huòzhě shǐyòng qiángzhì lèixíng zhuǎnhuàn (int), huòzhě bǎ foo2 de fǎnhuí lèixíng gǎi wéi int.

Task 3 — Expected values while debugging

Line	c	d	Note / Not / 说明
c = 1; d = 1.4;	1	1.4	start
foo1();	1	1.4	return value 1.0 ignored
c = foo2(10,20);	30	1.4	F11 enters foo2: its local c = 100, main's c NOT affected
d = foo2(1,2);	30	3.0	
d = c;	30	30.0	int → double, no warning (widening)

Task 4 — Let func2 change main's c (3 ways)

EN A function receives a **copy**, so it cannot change the caller's variable directly. Three solutions:

TR Fonksiyon bir **kopya** alır; bu yüzden çağıranın değişkenini doğrudan değiştiremez. Üç çözüm:

中文 函数得到的是实参的副本，不能直接修改主函数的变量。三种解决方法：

Hánshù dédào de shì shícān de fùběn, bù néng zhíjiē xiūgǎi zhǔ hánshù de biànliàng. Sān zhǒng jiějué fāngfǎ:

```
// (A) Global variable / Global değişken / 全局变量 (quánjú biànliàng)
int c;                          // outside all functions
void func2() { c = 100; }       // changes the one and only c
int main() { c = 1; func2(); /* c == 100 */ }

// (B) Pointer / İşaretçi / 指针 (zhǐzhēn)  — works in C and C++
void func2(int *p) { *p = 100; }
int main() { int c = 1; func2(&c); /* c == 100 */ }

// (C) Reference / Referans / 引用 (yǐnyòng) — C++ only
void func2(int &r) { r = 100; }
int main() { int c = 1; func2(c); /* c == 100 */ }
```

EN Also: **return** the new value (`c = func2(c);`) is a fourth, clean option. Global variables work but are considered bad style in large programs.

TR Ayrıca yeni değeri **return** ile döndürmek (`c = func2(c);`) dördüncü temiz yoldur. Global değişkenler çalışır ama büyük programlarda kötü stil sayılır.

中文 另外，也可以用 return 返回新值（c = func2(c);）。全局变量虽然可行，但在大型程序中不推荐。

Lìngwài, yě kěyǐ yòng return fǎnhuí xīn zhí (c = func2(c);). Quánjú biànliàng suīrán kěxíng, dàn zài dàxíng chéngxù zhōng bù tuījiàn.

5. Complete corrected program / Tam düzeltilmiş program / 完整修改后的程序

```
#include <iostream>
using namespace std;
void   foo1();
double foo2(double a, double b);
void   func2(int *p);
int main() {
    int c = 1;
    double d = 1.4;
    foo1();
    c = (int)foo2(10, 20);
    d = foo2(1, 2);
    func2(&c);
    cout << "c=" << c << " d=" << d << endl;
    return 0;
}
void foo1() { }
double foo2(double a, double b) { int c = 100; (void)c; return a + b; }
void func2(int *p) { *p = 100; }
```

Output / Çıktı / 输出: c=100 d=3 (tested with g++ -Wall -Wconversion: 0 errors, 0 warnings)

6. Vocabulary / Sözlük / 词汇表

中文	Pinyin	English	Türkçe
函数	hánshù	function	fonksiyon
定义	dìngyì	definition	tanım
调用	diàoyòng	call / invoke	çağırma
声明	shēngmíng	declaration	bildirim
返回值	fǎnhuízhí	return value	dönüş değeri
参数	cānshù	parameter	parametre
形式参数	xíngshì cānshù	formal parameter	biçimsel parametre
实际参数	shíjì cānshù	actual parameter / argument	gerçek parametre / argüman
全局变量	quánjú biànliàng	global variable	global değişken
局部变量	júbù biànliàng	local variable	yerel değişken
作用域	zuòyòngyù	scope	kapsam
同名变量	tóngmíng biànliàng	same-name variable	aynı adlı değişken
编译器	biānyìqì	compiler	derleyici
编译	biānyì	compile	derlemek
警告	jǐnggào	warning	uyarı
错误	cuòwù	error	hata
整型	zhěngxíng	integer type (int)	tam sayı tipi
浮点型	fúdiǎnxíng	floating-point type	kayan noktalı tip
类型转换	lèixíng zhuǎnhuàn	type conversion	tip dönüşümü
调试	tiáoshì	debugging	hata ayıklama
逐过程	zhú guòchéng	step over (F10)	adım atla
逐语句	zhú yǔjù	step into (F11)	içine adımla
断点	duàndiǎn	breakpoint	kesme noktası

中文	Pinyin	English	Türkçe
主函数	zhǔ hánshù	main function	ana fonksiyon
子程序	zǐ chéngxù	subroutine	alt program
指针	zhǐzhēn	pointer	işaretçi
引用	yǐnyòng	reference	referans
内存分配	nèicún fēnpèi	memory allocation	bellek ayırma
实验	shíyàn	experiment / lab	deney / laboratuvar

7. Self-check / Kendini test et / 自测题

English	Türkçe	中文	Answer
Does a declaration have a { } body?	Bildirimin { } gövdesi var mı?	声明有 {} 部分吗？	No / Hayır / 没有 (méiyǒu)
If foo is defined below main, what do you need?	foo, main'in altında tanımlıysa ne gerekir?	如果函数定义在 main 后面，需要什么？	A prototype / Prototip / 函数声明
Does foo2's local c change main's c?	foo2'nin yerel c'si main'in c'sini değiştirir mi?	foo2 的局部变量 c 会影响 main 的 c 吗？	No / Hayır / 不会 (bú huì)
Which key enters a function while debugging?	Hata ayıklarken fonksiyonun içine giren tuş?	调试时哪个键进入函数内部？	F11
Why does int c = foo2(10,20) warn?	int c = foo2(10,20) neden uyarı verir?	为什么会产生警告？	double → int, data loss / veri kaybı / 数据丢失